Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Strategy kalıbı: amaç ve örnek (ekrandan, 00:10:00)
  - Amaç: bir işi yapan algoritmayı çalışma anında, o an hangisi daha uygunsa ona geçerek dinamik biçimde değiştirebilmek.
  - Örnek: bir metin editöründe satır sonuna gelince alt satıra geçme (sözcük kaydırma) algoritmaları. Hoca WordPad'de "kaydırma yok", "pencereye kaydır", "cetvele kaydır" seçeneklerini gösterdi; heceden kesme işaretiyle bölme de ayrı bir algoritmadır.
  - İlke: "neyin değiştiğini bul ve sarmala". Değişen şey satır bölümleme algoritması olduğu için o sarmalanır.
  - Slayttaki şema:

```
Uygulama {bölmeleyici: SatırBölümleme; metinDüzenle()}
   metinDüzenle() içinde: bölmeleyici.bölmele()
Uygulama ◇⟶ SatırBölümleme {bölmele()}
SatırBölümleme ◁— KesmeliBölümleyici {bölmele()}
                ◁— KelimedenBölümleyici {bölmele()}
                ◁— SınırdanBölümleyici {bölmele()}
```

  - Kullanıcı ayarlardan kesmeli bölmeyi seçerse uygulamanın `bölmeleyici` üyesine bir `KesmeliBölümleyici` nesnesi atanır; kelimeden bölmeyi seçerse `KelimedenBölümleyici` atanır.
- Strategy kalıbının yapısı ve gerçekleme kararları
  - Context, algoritmanın arayüzünü (bir ya da birden çok metot imzası) tanımlayan soyut Strategy sınıfıyla ilişkilidir; somut stratejiler bu arayüzü gerçekler.
  - Strategy bir arayüz olabilir. Strateji, Context'in başka verilerine ulaşmak zorundaysa stratejiden Context'e bir bağlantı gerekir; bu durumda soyut üst sınıf daha uygundur. Varsayılan bir algoritma varsa üst sınıftaki metot somut da olabilir, alt sınıflar onu ezer.
  - Algoritmanın birden çok adımı varsa ve bu adımların sırasına Context müdahale edecekse birden çok metot olur; Context yalnız "veriyi al, işi yap, sonucu döndür" diyecekse tek metot yeterlidir.
  - Hangi algoritmanın uygun olduğuna kim karar verir: Context, soyut Strategy (eklenecek somut bir metotla) ya da Context'i kullanan bir istekçi (Context'e somut stratejiyi verir). Kararın stratejide verilmesi Context'i somut gerçeklemelerden daha bağımsız yapar.
  - Kullanım anları: bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda; istekçilerin algoritmanın gerçeklenmesini bilmemesi gerektiğinde. Slayttaki "yalnız davranışlarında farklılık gösteren birçok ilişkili sınıf bulunduğunda" maddesini hoca kalıtımın genel tanımına uyduğu için beğenmedi ve slayttan çıkardı.
  - Zayıf yönler: istekçiler (ya da karar veren Context) değişik stratejilerin varlığından haberdar olmak zorunda kalabilir. Ortak arayüz yüzünden Context, bazı stratejilerin hiç kullanmayacağı verileri de aktarmak zorunda kalabilir; aynı bellek alanında sorun değildir ama dağıtık sistemde ya da servislerde ek yük olur.
- Strategy ile Builder karşılaştırması (ekrandan, 00:20:00)
  - Sınıf şeması olarak iki kalıp hemen hemen aynıdır. Fark, yapılan işte ve işin seçilme biçimindedir.
  - Builder'da amaç karmaşık bir varlığın parçalarının nasıl oluşturulacağına esnek biçimde karar vermektir; istemci (Director) aynı somut builder'a sürekli "şimdi şunu

yap” diye çağrı yapar ve builder parçaları nasıl birleştireceğini bilip denetler.
  - Strategy’de Context somut stratejiyi bir kez verir ve aradan çekilir; belli bir anda yalnız bir strateji etkindir, geçişler seyrektir. Soyut strateji, somut algoritmaların işi nasıl yaptığını bilmez.
  - Genel ders: sınıf şeması aynı olan kalıplar olabilir; “fark ayrıntıda gizlidir”.
- Strategy örneği: kişisel finans programında vergi hesabı (ekrandan, 00:23:30, 00:26:10)
  - Problem: uluslararası pazara hitap eden bir kişisel finans programı; Türkiye, ABD ve Avrupa Birliği’nin vergi kuralları birbirinden çok farklıdır. Ana program `TaskController`. Kaynak: Design Patterns Explained.
  - Çözüm 1 (switch-case):

```
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code) {
        this.code = code;
    }
    public double taxAmount(double income) {
        switch (code) {
            case TR: return income * 0.18;
            case EU: return income * 0.15;
            case US: return income * 0.12;
            default: return income * 0.10;
} } }
```

  - Örnekte yalnız oran değiştiği için gerçekte tek bir oran alanı yeterdi; hoca her `case`’in yüzlerce satırlık, ülkeye özgü karmaşık bir hesap olduğunun düşünülmesini istedi.
  - Çözüm 2 (Strategy): `«interface» TaxRule { + taxAmount(income: double): double }`; `TRrules`, `EUrules`, `USrules` bunu gerçekler. `TaskController`’da `tax: TaxRule` üyesi ve `setTax(tax: TaxRule)` vardır.

```
package strategy.solution2;
public interface TaxRule {
    public double taxAmount( double income );
}
public class TaskController {
    private TaxRule tax;
    public void setTax(TaxRule tax) { this.tax = tax; }
    public double taxAmount( double income ) {
        return tax.taxAmount( income );
    }
}
public class TRrules implements TaxRule {
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

  - Kazanç: 400 satırı aşan tek bir dev sınıf yerine 110-120 satırlık üç-dört sınıf. Toplam satır sayısı pek değişmez; önemli olan her işin tek bir yerde yapılmasıdır. İyi tasarım, problem hangi düzeyde parçalara ayrılabiliyorsa o düzeyde parçalanmalıdır.
  - Başka ülkeye bağlı işler (ör. gelire göre değişen zorunlu bireysel emeklilik katkısı) eklenecekse hoca aynı stratejiye yeni metot eklemek yerine ayrı bir strateji (ör. `SavingsRule`) kurmayı tercih eder; tek stratejiye iki sorumluluk yüklemek uyumu (cohesion) düşürür.
  - Strategy, Singleton ile birlikte kullanılabilir: ör. Türkiye kuralları sistem genelinde değişmediği için `TRrules` tek nesneli olmaya adaydır, ama şart değildir.
- State kalıbı (ekrandan, 00:34:40)
  - Amaç: nesnenin iç durumu değişince davranışının da büyük ölçüde (çalışma biçimini tamamen) değişmesi.
  - GoF örneği: bir TCP bağlantısı belli bir anda Established, Listen ya da Closed durumundadır; `Open`, `Close`, `Acknowledge` istekleri her durumda çok farklı çalışır. Switch-case yerine durumlar sarmalanır; bağlantı ile durumu ayrı nesnelerdir (parça-bütün

ilişkisi).

`TCPConnection {Open(), Close(), Acknowledge()} ◇⟶state⟶► TCPState {Open(), Close(), Ack`
`Open() içinde: state->Open()`
`TCPState ◁⟵ TCPEstablished, TCPListen, TCPClosed (her biri Open, Close, Acknowledge)`

- Durum geçişlerinden genellikle Context sorumludur, çünkü "ne olunca öbür duruma geçilecek" bilgisinin çoğu Context'tedir; geçiş state içine kodlanırsa state bunu sürekli Context'e sormak zorunda kalır.
- Kullanım anları: bir nesnenin çalışma şeklinin çalışma anında durumuna göre değişebileceği yerler; nesnenin durumuna bağlı, karmaşık ve uzun, çok kısımlı karar verme komutlarının gerektiği anlar.

- State ile Strategy farkı (ekrandan, 00:40:00)
  - UML şemaları aynıdır. State'te Context'teki durum değiştikçe Context kendi ConcreteState'ini seçer; geçişler açıkça yapılır (Context'teki komutlarla ya da ConcreteState'ler içinden) ve daha sıktır.
  - Strategy'de geçişler seyrektir (programın bütün çalışması boyunca hiç değişmeyebilir); strateji nesneleri Context hakkında daha az şey bilir, kendilerine gelen parametrelere göre çalışır. State nesneleri Context'i daha yakından izler.
  - Benzetme: strateji uzun vadeli oyun planıdır, taktik anlık değişir. Maçta savunma stratejisi seçildiyse rakibe göre taktikler değişir; strateji ancak devre arası gibi seyrek anlarda değişir.
- State örneği: PIM yazılımında Temiz/Kirli durumu (ekrandan, 00:50:00, 00:48:20)
  - Kişisel bilgi yöneticisinde (PIM) kayıtlar topluca iki durumdan birindedir: Clean ya da Dirty. Bir kayıt bile değiştirilip kaydedilmediyse sistem kirlidir; kaydedilince temize döner (Eclipse'te kaydedilmemiş dosyanın başındaki yıldız gibi). Kaynak: Applied Java Patterns.

```
State {+ edit(); + save(f: File, s: Serializable)}
State ◁⟵ CleanState {+ edit(); + save(f: File, s: Serializable)}
State ◁⟵ DirtyState
PIMApplication {+ CLEAN: State; + DIRTY: State; - currentState: State;
                + getCurrentState(): State; + setCurrentState(state: State)}
PIMApplication ⟶► State, CleanState, DirtyState
```

  - `CLEAN` ve `DIRTY` public, static ve final üyelerdir; uygulama temiz durumda başlar. Asıl kaydetme işi iki durumda da aynı olduğu için `save` State üst sınıfında tam metot olarak kodlanır; durumlar arasındaki tek fark geçiştir.
  - `CleanState` (hoca derste düzeltti): `edit` önce durumu kirliye çevirir sonra `super.edit()` çağırır; `save` önce `super.save(f, s)` ile kaydeder, ancak ondan sonra durumu temize çevirir (kaydetme başarısız olursa yanlışlıkla temize geçilmesin diye).

```
public class CleanState extends State {
    public void edit( ){
        StateFactory.setCurrentState(PIMApplication.DIRTY);
        super.edit();
    }
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
        StateFactory.setCurrentState(PIMApplication.CLEAN);
    }
}
public class DirtyState extends State { }
```

  - Ders sonrası not: slayttaki kod geçişi `StateFactory.setCurrentState(...)` ile yapıyor, sınıf şemasında ise `setCurrentState` `PIMApplication`'da görünüyor; büyük olasılıkla kodda eski bir sınıf adı kalmıştır. Slaytta `DirtyState`'in üstündeki paket adı da `dp.flyweight.example` yazıyor; büyük olasılıkla kopyalama hatasıdır, doğrusu `dp.state.example` olmalıdır.
  - `DirtyState` üst sınıfla aynıdır, yalnız somuttur; böyle sınıflara işaretleyici (marker) sınıf denir. Bu iş daha çok arayüzlerle yapılır; hoca örneği bu yüzden pek beğenmediğini söyledi.
  - Derste `PIMApplication`'a `edit()` ve `save(File f, Serializable s)` metotları eklendi; ikisi de işi `currentState` nesnesine yaptırır.
- Template Method kalıbı (ekrandan, 01:10:00, 01:14:40)

  - Amaç: bir algoritmanın bazı adımlarını değiştirilemez biçimde belirleyip bazılarının ayrıntılarının belirlenmesini ertelemek (alt sınıflara bırakmak).
  - Örnek: farklı belge türleriyle çalışacak uygulamalar için bir çerçeve program (framework). Çerçeve Belge (`Document`) ve Uygulama (`Application`) sınıflarını sunar; kullanıcı programcılar bunlardan kalıtımla kendi sınıflarını türetir. Bazı adımlar her belge türü için ortaktır ve belli sırayla, unutulmadan yapılmalıdır.

```
Document {Save(), Open(), Close(), DoRead()}   ◁── MyDocument {DoRead()}
Application {AddDocument(), OpenDocument(), DoCreateDocument(),
             CanOpenDocument(), AboutToOpenDocument()}   ◁── MyApplication
Application ◇──docs──► Document
MyApplication.DoCreateDocument(): return new MyDocument
```

```
void Application::OpenDocument (const char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        _docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

  - `OpenDocument` şablon metottur: açılabilir mi bak, belgeyi oluştur, listeye ekle, açılmadan önceki ilk işlemleri yap, aç ve oku. Değişen adımlar (`CanOpenDocument`, `DoCreateDocument`, `AboutToOpenDocument`, `DoRead`) soyut metottur.
  - Şablon metot alt sınıfların ezemeyeceği biçimde tanımlanmalıdır: Java'da `final`. Alt sınıfa bırakılan metotlar Java'da `abstract`, C++'ta saf sanal (pure virtual) olmalıdır.
  - Kullanım anları: değişmez kısımları bir kez kodlayıp değişen kısımları alt sınıflarda gerçeklemek; alt sınıfların davranışını denetlemek; tutarsızlığı ve kod tekrarını önlemek için kardeş sınıfların ortak davranışını üst sınıfta toplamak.

- Template Method örneği: farklı veritabanlarından kayıt çekme (ekrandan, 01:25:00)
  - Beş adım: (1) bağlantı komutunu biçimlendir, (2) bağlantı komutunu gönder, (3) select komutunu biçimlendir, (4) select komutunu gönder, (5) dönen veri kümesini işle. 1. ve 3. adımlar veritabanı yönetim sistemine göre değişir; 2, 4 ve 5 ortaktır. Kaynak: Design Patterns Explained.

```
QueryControl {- qt: QueryTemplate; + QueryControl(QueryTemplate)}  1 ──► 1  «abstract» Qu
QueryTemplate {+ doQuery(DBname, querySpec: String) {final};
               ~ formatConnect(DBname: String) {abstract};
               ~ formatSelect(querySpec: String) {abstract}}
QueryTemplate ◁── OracleQT, SQLServerQT, GraphDBQT
```

  - Biçimlendirme metotları iç çalışmayla ilgili olduğu için protected tanımlanmıştır; bir kütüphane hazırlanmadığı için Java'da paket görünürlüğü (~) de kullanılabilir.

- Visitor kalıbı (ekrandan, 01:30:00, 01:40:00, 01:50:00)
  - Amaç: bir nesne yapısını oluşturan elemanlar üzerinde yapılacak işlemler tanımlamak. Sorun: yapı her türden bağıntılar içeren sınıfların nesnelerinden kurulmuştur; elemanların sınıflarını sürekli değiştirmek istemeyiz ama zamanla yeni işlemler eklemek isteriz.
  - Eleştiri: daha yapılacak işlemler varsa sorumluluklar tam belirlenmemiştir; sınıf yapısını değiştirip farklı tasarım yapmak daha doğru olabilir. Zaman baskısıyla erken teslim gibi durumlarda kalıp yine de işe yarar.
  - GoF örneği: derleyici kaynak kodu farklı türden düğümlere (atama, değişken referansı, aritmetik işlem) ayırır; düğümler üzerinde tip denetimi, kod üretimi, güzel yazdırma (pretty printing) gibi işlemler yapılır, ama her işlem her düğümde anlamlı değildir.
  - Hocanın çizdiği `x = y + z` ağacı: kökte atama düğümü; solda `x` değişken referansı, sağda toplama düğümü ve onun altında `y` ve `z`. Atama düğümünde tip denetimi gerekmez, toplamanın çocuklarında gerekir (ikisi de tamsayıysa toplama, biri `String` ise birleştirme).

```
NodeVisitor {VisitAssignment(AssignmentNode); VisitVariableRef(VariableRefNode)}
NodeVisitor ◁── TypeCheckingVisitor, CodeGeneratingVisitor
Program ◇──► Node {Accept(NodeVisitor)}
Node ◁── AssignmentNode {Accept(NodeVisitor v)}:  v->VisitAssignment(this)
Node ◁── VariableRefNode {Accept(NodeVisitor v)}: v->VisitVariableRef(this)
```

  - Önce bir tip denetimi ziyaretçisi bütün düğümleri dolaşır; hata yoksa kod üretimi ziyaretçisi dolaşır. Böylece if-else ve switch-case'ten kurtulunur.
  - Zayıf yönler: düğüm, ziyaretçinin hangi metodunun kendi türü için olduğunu bilmek zorundadır (ek bağlaşım; yine de ziyaretçinin somut türüne bağımlılık yoktur). Yeni bir somut eleman türü eklemek zordur: Visitor arayüzüne ve bütün somut ziyaretçilere yeni metot eklenmesi gerekir. Bir işlem bir düğüm türünde anlamsızsa boş gövdeli metot yazılır.
  - Etkileşim (dizi şeması): nesne yapısı her elemana `Accept(aVisitor)` der; eleman ziyaretçinin kendi türüne ait `VisitConcreteElementX(this)` metodunu çağırır; ziyaretçi elemanın `OperationX()` metodunu çağırır; sonra sıradaki elemana geçilir.
  - Kullanım anları: değişen arayüzlere sahip nesnelerden oluşan bir yapıda farklı işlemler yapılması gerektiğinde VE bu yapıdaki türler değişmediğinde ya da ender değiştiğinde.
- Visitor örneği: proje yönetim yazılımı (ekrandan, 02:10:00, 02:17:40, 02:30:00, 02:33:30, 02:40:00)
  - Varlıklar: `Project`, `Task`, `Deliverable` (ve şemada gösterilmeyen `DependentTask`); hepsi «`Abstract`» `ProjectItem`'dan türer (ad ve açıklama). `Task` alt öğeler (`ArrayList<ProjectItem>`), `timeRequired` ve bir `Contact` sahibi tutar; `Deliverable` `materialsCost` ve `productionCost` tutar; zamanı yoktur. Görevin maliyeti yok, ürünün süresi yok. Kaynak: Applied Java Patterns.
  - Çözüm 1: `ProjectCalculator`, proje öğelerini dolaşıp `instanceof` ile türüne bakar; süre için `Task`'ların `timeRequired` değerlerini, maliyet için `Deliverable`'ların malzeme ve üretim maliyetini toplar. Alt görevler özyinelemeli private yardımcı metotlarla (`calculateTimeRequired(Task)`, `calculateCost(Task)`) dolaşılır. Hoca birbirine çok benzeyen bu kodu kopyala-yapıştır kodu olarak eleştirdi.
  - `instanceof`'tan kurtulmak için `getCost` ve `getTimeRequired`'ı `ProjectItem`'a koymak kalıtım ilkesine ters düşer: görev için sıfır döndüren anlamsız (dummy) metotlar gerekir.
  - Çözüm 2 (Visitor): `ProjectVisitor` arayüzü (`visitDependentTask`, `visitDeliverable`, `visitTask`, `visitProject`); `ProjectCostVisitor` ve `ProjectTimeVisitor` bunu gerçekler. Her öğeye `accept(ProjectVisitor v)` eklenir: `Project` içinde `v.visitProject(this)`, `Task` içinde `visitTask`, `Deliverable` içinde `visitDeliverable` çağrılır. Başka değişiklik yoktur.
  - Ziyaretçi yapıcısında `project.accept(this)` ile dolaşımı başlatır. `ProjectCostVisitor.visitDeliverable` içinde `cost += p.getMaterialsCost() + p.getProductionCost();` yapılır; `visitTask` alt öğeleri dolaşır. `ProjectTimeVisitor.visitTask` içinde `time += p.getTimeRequired();` yapılır ve alt öğelere `item.accept(this)` denir; `visitDeliverable` boştur.
  - Örnek veri: "Yüksek Lisans" projesi; her dönemin dersleri 15 haftalık bir görev, her ders 45 saat ders ve 30-40 saat sınav çalışması maliyetli bir ürün. İki çözüm de aynı sonucu verdi: proje süresi 60 hafta, maliyeti 1174 saat çalışma.
  - Kazanç: `instanceof` denetimlerinden kurtulma; yeni bir hesap ya da rapor için yalnız yeni bir ziyaretçi eklemek yeterli. Kayıp: zaman ziyaretçisindeki boş `visitDeliverable` gibi boş gövdeli metotlar ve yeni bir düğüm türü eklemenin maliyeti.

Hocanın Özellikle Vurguladığı Kısımlar

- Kendinizi switch-case kullanırken görüyorsanız, çok biçimliliğin uygun bir kullanımına ihtiyacınız var demektir.
  - Hem Strategy (vergi kuralları) hem State (TCP durumları) örneği switch-case'li çözümden çok biçimliliğe geçişle anlatıldı.
- Sınıf şeması aynı olan kalıplar vardır; fark amaçta ve ayrıntıdadır.
  - Strategy ile Builder, Strategy ile State şema olarak aynıdır. State'te Context durumu bilir, durumlar arası kendisi geçer ve geçişler sıktır; Strategy'de geçişler seyrektir.

- Bir problem hangi düzeyde parçalara ayrılabiliyorsa tasarım da o düzeyde parçalara ayrılmalıdır.
  - Farklı ülkelerin farklı kuralları varsa her ülkenin kuralı ayrı bir parça (strateji) olmalıdır.
- Farklı sorumlulukları tek stratejiye yığmak yerine her sorumluluk için ayrı strateji kalıbı kurmak uyumu artırır.
- Template Method'da şablon metot `final`, alt sınıflara bırakılan adımlar `abstract` (C++'ta saf sanal) olmalıdır.
  - Böylece çerçeveyi kullanan programcı adımların sırasını bozamaz ya da bir adımı atlayamaz.
- Veritabanı örneğinde "State" demek yanlış, "Strategy" yakın, "Template Method" en uygun cevaptır.
  - Durum değişimi yoktur; farklı erişim biçimleri vardır, ama ortak adımlar da vardır.
- Visitor, nesne yapısındaki türler değişmiyorsa ya da ender değişiyorsa kullanılmalıdır.
  - Tasarım kalıpları değişiklikleri en az angaryayla karşılamaya yönelik çözümlerdir; yeni eleman türü eklemek Visitor'da çok iş çıkarır.
- Hoca `instanceof`'u "gerekli kötülük" (necessary evil) olarak görür; gerekebilir ama mümkün olduğunca kaçınılmalıdır.
- Hoca Visitor'u karmaşıklığı nedeniyle pek tercih etmediğini söyledi; yeni katılan ve kalıbı bilmeyen bir programcının uyumu zor olabilir, bu yüzden kullanmamak da savunulabilir bir tercihtir.

Kısa Tekrar Notları

- Strategy: algoritmayı nesneye sar, Context'e ata, çalışma anında değiştir. Kim seçer: Context, soyut strateji ya da istemci.
- Strategy zayıflıkları: istekçi stratejileri bilmek zorunda kalabilir; ortak arayüz yüzünden gereksiz veri aktarılabilir (dağıtık sistemde ek yük).
- Strategy ve Builder: şema aynı. Builder'da istemci sürekli adım adım çağrı yapar; Strategy'de strateji bir kez verilir.
- Vergi örneği: `TaxRule` arayüzü, `TRrules/EUrules/USrules`; stratejiler Singleton olmaya adaydır.
- State: iç durum değişince davranış tamamen değişir. TCP: Established/Listen/Closed. Geçişleri genellikle Context yapar ve geçişler sıktır.
- State ve Strategy: State Context'i yakından izler; strateji yalnız kendisine verilen parametreyle çalışır. Strateji uzun vadeli plan, taktik anlıktır.
- PIM örneği: `CleanState.edit` önce DIRTY'ye geçer sonra `super.edit()`; `save` önce kaydeder sonra CLEAN'e geçer. `DirtyState` işaretleyici sınıftır.
- Template Method: değişmez adımlar üst sınıfta `final` şablon metotta; değişen adımlar `abstract`. Örnekler: `Application::OpenDocument`, `QueryTemplate.doQuery`.
- Veritabanı sorgusu: 5 adımdan 1 ve 3 değişir (`formatConnect`, `formatSelect`), 2, 4, 5 ortaktır.
- Visitor: elemanlar `Accept(visitor)` sunar, eleman `visitor.VisitX(this)` çağırır (çift yönlendirme). Yeni işlem eklemek kolay, yeni eleman türü eklemek zordur.
- Proje örneği: `ProjectCostVisitor` ve `ProjectTimeVisitor`; `instanceof` kalktı, boş `visitDeliverable` geldi. Sonuç: 60 hafta, 1174 saat.

Detaylı Açıklamalar

- Strategy kalıbı, kalıtımın getirdiği çok biçimlilik esnekliği üzerine kurulmuştur. Metin editörü örneğinde `Uygulama` sınıfı metni düzenlerken satır bölme işini sahip olduğu `SatırBölümleme` nesnesine yaptırır; kullanıcı ayarı değiştikçe bu üyeye başka bir somut bölümleyici atanır. Kalıbın asıl tasarım kararları, arayüzün tek metot mu çok metot mu olacağı, stratejinin Context'e geri bağlantıya ihtiyaç duyup duymadığı ve uygun algoritmayı kimin seçeceğidir. Seçimi Context yaparsa Context bütün somut stratejileri tanımak zorunda kalır; farklı Context sınıfları farklı strateji alt kümelerine ihtiyaç duyabilir (ör. biri A ve B, öbürü C ve D). Seçimi istemci yaparsa algoritma Context'e tamamen şeffaf olur.
- Vergi örneğinde ilk çözümde `TaskController` bir `Country` enum'una göre switch-case ile vergi hesaplar. Gerçekte her ülkenin kuralı yüzlerce satırlık bir koddur ve hepsi tek sı-

nıfta toplanır. Strategy ile her ülkenin kuralı `TaxRule` arayüzünü gerçekleyen ayrı bir sınıfa taşınır; `TaskController` yalnız `tax.taxAmount(income)` çağırır. Toplam satır sayısı pek azalmaz, ama her iş tek bir yerde yapılır. Ülkeye bağlı başka bir iş (ör. emeklilik katkısı) gelirse aynı stratejiye metot eklemek yerine ayrı bir strateji kurmak her stratejiyi tek sorumluluğa odaklar. Ülke kuralları değişmediği için somut stratejiler Singleton olarak da yazılabilir.

- State kalıbı şema olarak Strategy'nin aynısıdır. Fark şudur: bağlantı o anda hangi durumda olduğunu ve hangi duruma geçeceğini bilir, kendini o duruma geçirir ve işi o durum nesnesine yaptırır. Durum geçişleri sıktır ve genellikle Context'te kodlanır, çünkü geçiş kararı için gereken bilgi oradadır. PIM örneğinde kaydetme işinin gövdesi üst sınıftadır, çünkü temiz ve kirli durumda kaydetme aynı biçimde yapılır; yalnız geçiş farklıdır. Hoca `CleanState.save` içindeki sırayı derste düzeltti: önce kaydetme, sonra temiz duruma geçiş. Kaydetme sırasında hata çıkarsa sistem yanlışlıkla temiz görünmemelidir. `edit` üst sınıfta kodlandığı için `CleanState.edit` sonunda `super.edit()` çağrılmalıdır; `edit` soyut bırakılsaydı aynı kod hem `CleanState` hem `DirtyState` içinde tekrar yazılmak zorunda kalınırdı.
- Template Method'da üst sınıf bir algoritmanın iskeletini somut ve `final` bir metotta kurar; değişen adımları soyut metotlar üzerinden çağırır. Belge çerçevesinde `OpenDocument`, belgenin türünden bağımsız adımları (açılabilirlik denetimi sonucuna göre durma, listeye ekleme, açma) kendisi yapar; türe bağlı adımları (`CanOpenDocument`, `DoCreateDocument`, `AboutToOpenDocument`, `DoRead`) alt sınıflara bırakır. Kalıp, kardeş sınıflarda benzer ama tam aynı olmayan işlemler bulunduğunda ortak kısmı üst sınıfa taşımanın adı konmuş hâlidir. Veritabanı örneğinde Strategy de kullanılabilirdi; ama ortak adımlar bulunduğu için en uygun cevap Template Method'dur.
- Visitor kalıbında her eleman `accept` metoduyla ziyaretçiyi kabul eder ve kendi türüne ait `visit` metodunu çağırır; böylece ziyaretçi elemanın türünü `instanceof` ile sormadan doğru işlemi yapar. Proje örneğinde süre ve maliyet hesapları iki ayrı ziyaretçiye ayrılmıştır. Öğe sınıflarına yalnız birer satırlık `accept` metodu eklenmiştir, raporlama kodu ise ziyaretçilere taşınmıştır. Hocanın değerlendirmesine göre kazanç çok büyük değildir: `instanceof` kalkmış, ama boş gövdeli metotlar ortaya çıkmıştır. Hoca boş gövdeli metotların kalıtımda "reddi miras" olmamasından kaynaklandığını ve bunun doğru bir dil kararı olduğunu söyledi. Ek maliyet az olduğu için nesneye dayalı ilkeleri daha az ihlal eden bu çözüm yapılmaya değer; yine de karmaşıklığı nedeniyle kullanmamak da meşru bir tercihtir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.